Deep Learning

with pytorch

2026 Spring

조상구교수

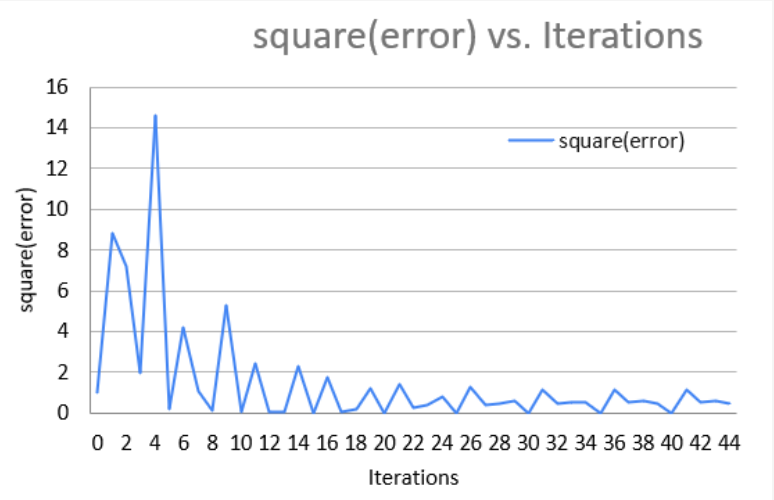
Learning PyTorch in 30 Min

1. SGD : recap
2. 5-step recipe
3. Tensor
4. Autograd
5. Backpropagation
6. Gradient Descent With Autograd and Backpropagation
7. Training Pipeline: Model, Loss, and Optimizer
8. Linear Regression

Stochastic Gradient Descent

모델 설정 > 예측 값 생성 > 손실함수 값 생성 > gradient 만큼 손실을 감소 > 반복 : Optimization Process

	A	B	C	D	E	F	G	H	I	J
1	learning rate		Data		model = y = a+bx			y_hat - y		
2	0.01	Iterations	x	y	절편 (α)	기울기 (β)	y_hat	= error	square(error)	absolute error
3	1 epoch	0	1	1	0.0000	0.0000	0.0000	-1.0000	1.0000	1.0000
4		1	2	3	0.0100	0.0100	0.0300	-2.9700	8.8209	2.9700
5		2	4	3	0.0397	0.0694	0.3173	-2.6827	7.1969	2.6827
6		3	3	2	0.0665	0.1767	0.5967	-1.4033	1.9694	1.4033
7		4	5	5	0.0806	0.2188	1.1746	-3.8254	14.6337	3.8254
8	2 epoch	5	1	1	0.1188	0.4101	0.5289	-0.4711	0.2219	0.4711
9		6	2	3	0.1235	0.4148	0.9531	-2.0469	4.1898	2.0469
10		7	4	3	0.1440	0.4557	1.9669	-1.0331	1.0673	1.0331
11		8	3	2	0.1543	0.4971	1.6455	-0.3545	0.1257	0.3545
12		9	5	5	0.1579	0.5077	2.6963	-2.3037	5.3070	2.3037
13	3 epoch	10	1	1	0.1809	0.6229	0.8038	-0.1962	0.0385	0.1962
14		11	2	3	0.1829	0.6248	1.4325	-1.5675	2.4569	1.5675
15		12	4	3	0.1985	0.6562	2.8233	-0.1767	0.0312	0.1767
16		13	3	2	0.2003	0.6633	2.1901	0.1901	0.0361	0.1901
17		14	5	5	0.1984	0.6575	3.4862	-1.5138	2.2917	1.5138
18	4 epoch	15	1	1	0.2135	0.7332	0.9468	-0.0532	0.0028	0.0532
19		16	2	3	0.2141	0.7338	1.6816	-1.3184	1.7381	1.3184
20		17	4	3	0.2273	0.7601	3.2678	0.2678	0.0717	0.2678
21		18	3	2	0.2246	0.7494	2.4729	0.4729	0.2236	0.4729
22		19	5	5	0.2199	0.7352	3.8961	-1.1039	1.2187	1.1039



5 Step Deep Learning Recipe

Iterate 5-steps until finding the parameters to minimize the loss function.

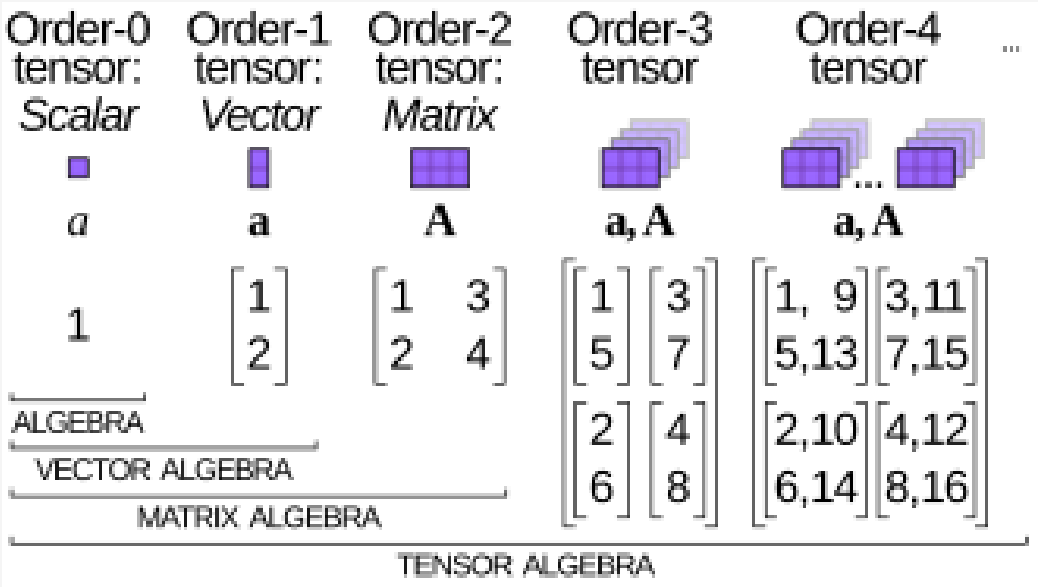
THE 5-STEP DEEP LEARNING RECIPE

Step	Concept (Math)	From Scratch (Raw Tensors)	Professional (`torch.nn`)
1. Prediction	Make a prediction: $\hat{y} = f(X, \theta)$	`y_hat = X @ W + b`	`y_hat = model(X)`
2. Loss Calc	Quantify the error: $L = \text{Loss}(\hat{y}, y)$	`loss = torch.mean((y_hat - y)**2)`	`loss = criterion(y_hat, y)`
3. Gradient Calc	Find the slope of the loss: $\nabla_\theta L$	`loss.backward()`	`loss.backward()`
4. Param Update	Step down the slope: $\theta_{t+1} = \theta_t - \eta \nabla_\theta L$	`W -= lr * W.grad`	`optimizer.step()`
5. Gradient Reset	Reset for the next loop.	`W.grad.zero_()`	`optimizer.zero_grad()`

tensor

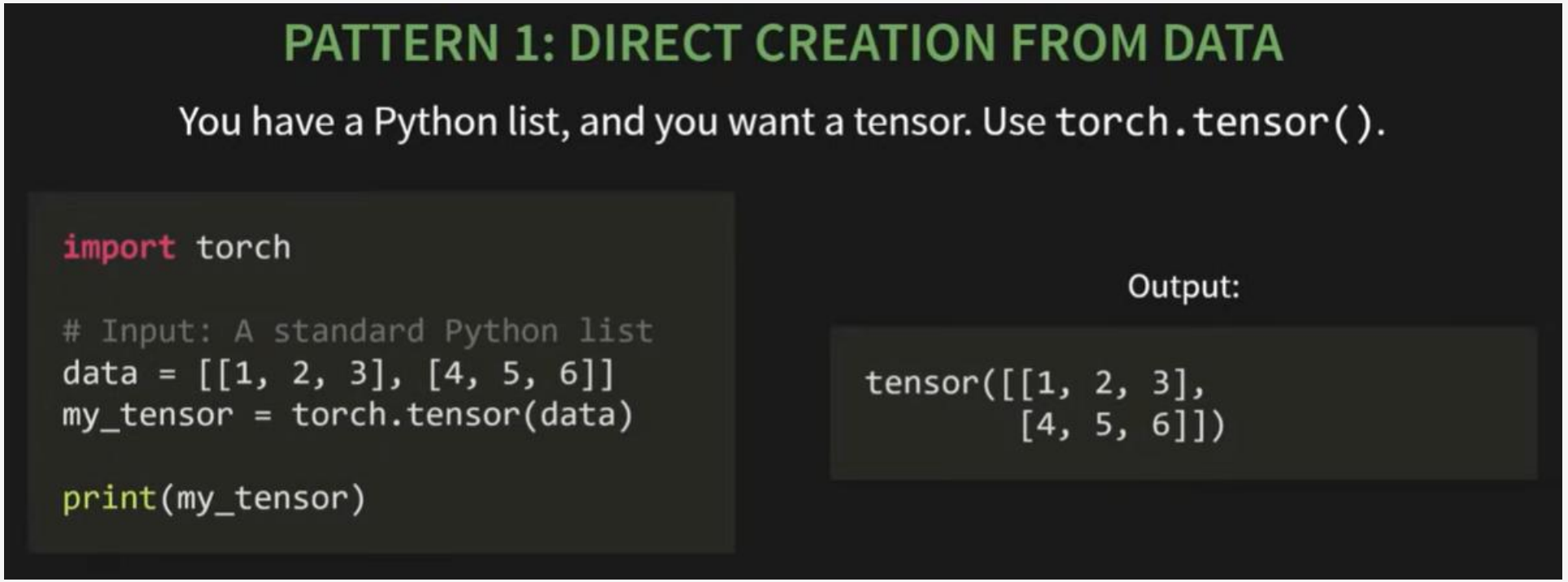
A torch.Tensor is a multi-dimensional matrix containing elements of a single data type.

IT'S THE BASIC "NOUN" OF THE PYTORCH LANGUAGE.

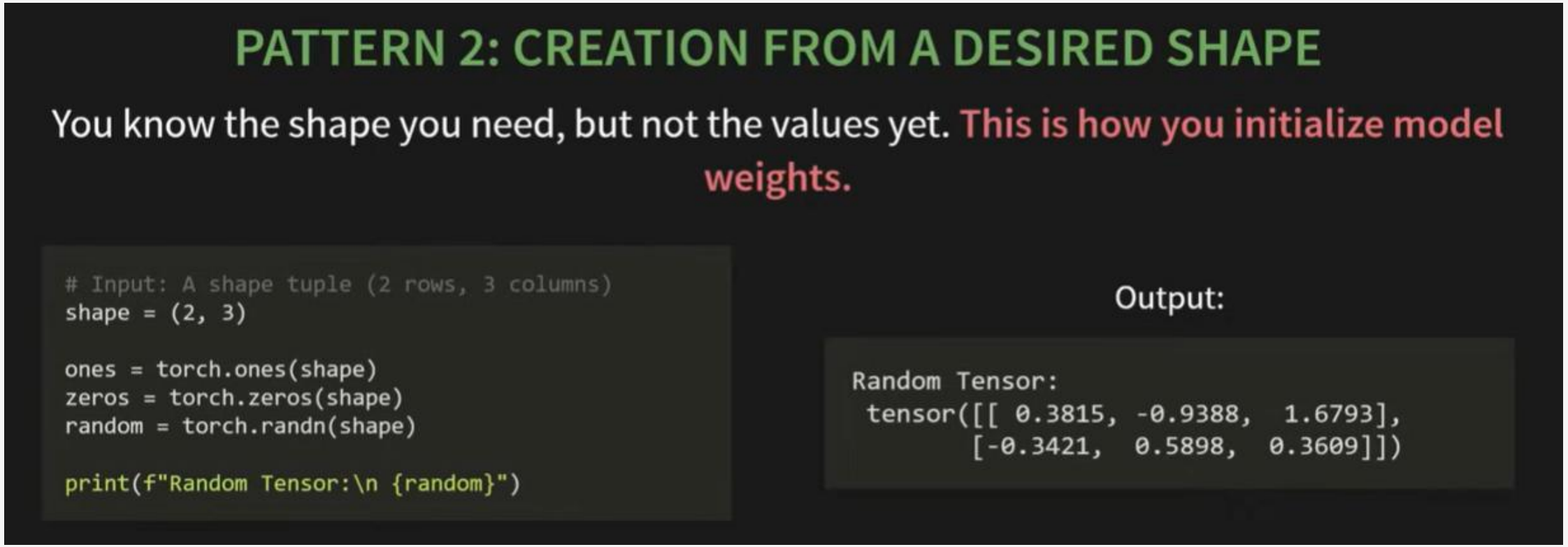


https://simple.wikipedia.org/wiki/Tensor

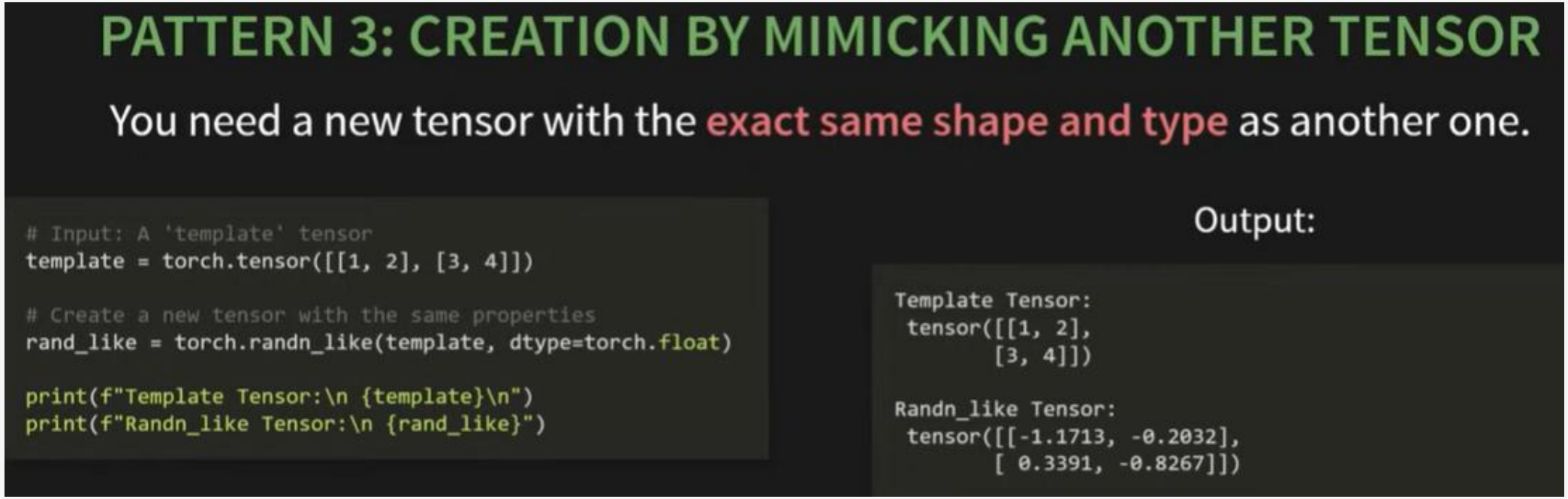
tensor


PATTERN 1: DIRECT CREATION FROM DATA
You have a Python list, and you want a tensor. Use torch.tensor().
import torch
# Input: A standard Python list
data = [[1, 2, 3], [4, 5, 6]]
my_tensor = torch.tensor(data)
print(my_tensor)
Output:
tensor([[1, 2, 3],
[4, 5, 6]])

tensor

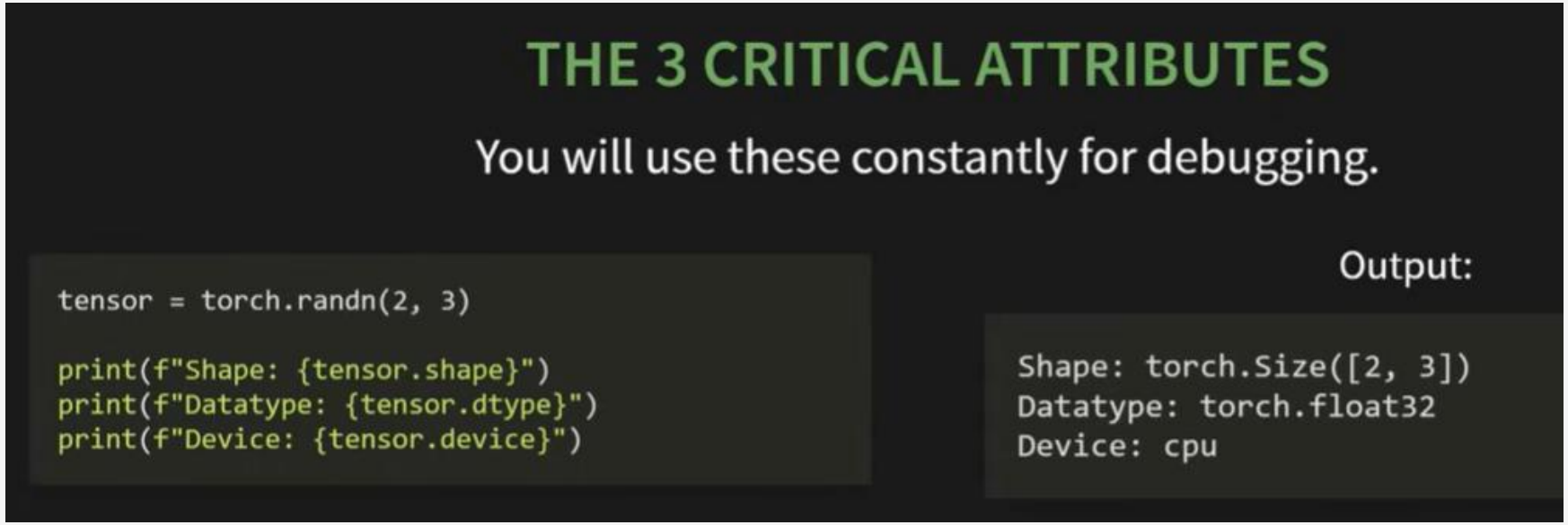

PATTERN 2: CREATION FROM A DESIRED SHAPE
You know the shape you need, but not the values yet. This is how you initialize model weights.
# Input: A shape tuple (2 rows, 3 columns)
shape = (2, 3)
ones = torch.ones(shape)
zeros = torch.zeros(shape)
random = torch.randn(shape)
print(f"Random Tensor:\n {random}")
Output:
Random Tensor:
tensor([[0.3815, -0.9388, 1.6793],
[-0.3421, 0.5898, 0.3609]])

tensor


PATTERN 3: CREATION BY MIMICKING ANOTHER TENSOR
You need a new tensor with the exact same shape and type as another one.
# Input: A 'template' tensor
template = torch.tensor([[1, 2], [3, 4]])
# Create a new tensor with the same properties
rand_like = torch.randn_like(template, dtype=torch.float)
print(f"Template Tensor:\n {template}\n")
print(f"Randn_like Tensor:\n {rand_like}")
Output:
Template Tensor:
tensor([[1, 2],
[3, 4]])
Randn_like Tensor:
tensor([[-1.1713, -0.2032],
[0.3391, -0.8267]])

tensor

tensor



`.shape`: A tuple describing the dimensions. Your #1 debugging tool.
90% of your errors in PyTorch will be shape mismatches.

`.device`: Where the tensor lives. cpu or cuda (GPU).

`.dtype`: The data type of the numbers. The default is `float32`.
This is not an accident.

WHY float32?

WHY NOT JUST INTEGERS?

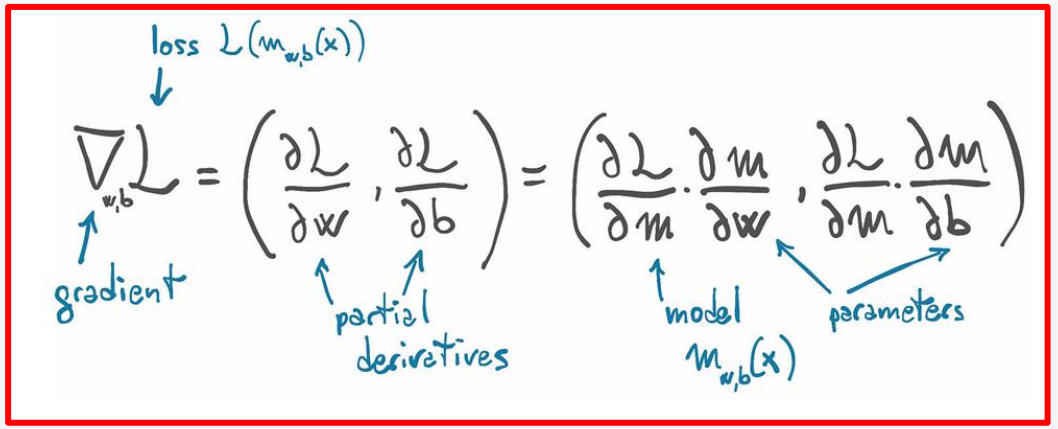
THE RULE IS SIMPLE:

- Model parameters (weights, biases) MUST be a float type. `float32` is the standard.
- Data that represents categories or counts can be integers.

tensor

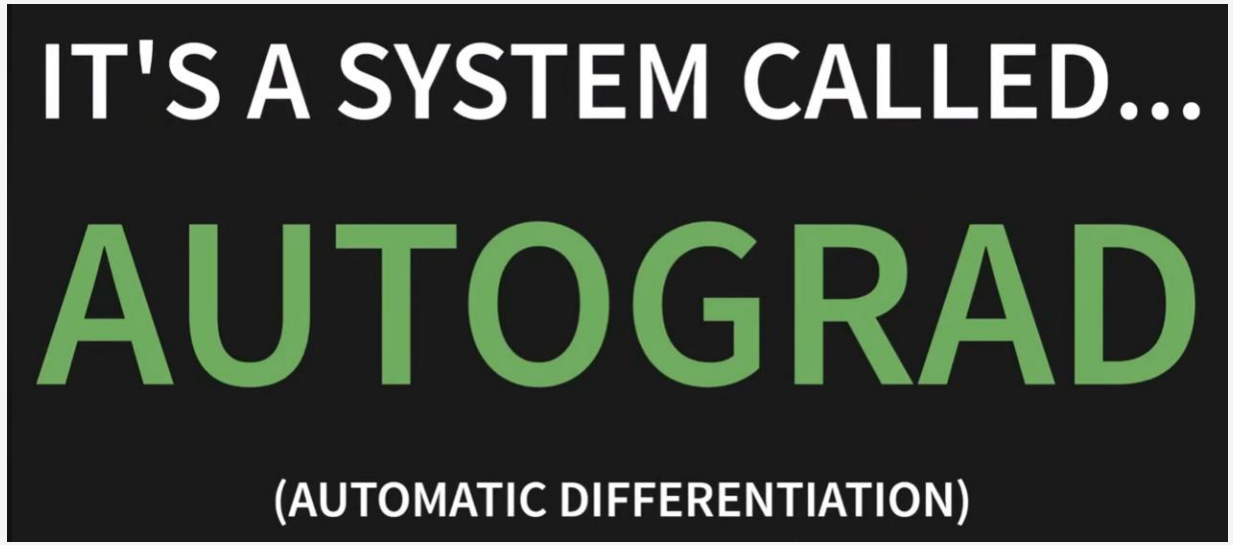
The derivative of the loss function with respect to the weights(float.32)

	Data		model = y = a+bx			y_hat - y	
Iterations	x	y	절편 (α)	기울기 (β)	y_hat	= error	square(error)
0	1	1	0.00000000000	0.00000000000	0.00000000000	-1.0000	1.0000
1	2	3	0.01000000000	0.01000000000	0.03000000000	-2.9700	8.8209
2	4	3	0.03970000000	0.06940000000	0.31730000000	-2.6827	7.1969
3	3	2	0.06652700000	0.17670800000	0.59665100000	-1.4033	1.9694
4	5	5	0.08056049000	0.21880847000	1.17460284000	-3.8254	14.6337



AUTOGRAD

The derivative of the loss function with respect to the weights(float.32)



구분	수식적 표현	의미
Differentiation	$\frac{d}{dx}(\dots)$	**행위:** "미분 연산자를 적용하는 과정"
Derivative	$\lim_{h\to 0}\frac{f(x+h)-f(x)}{h}$	**상태:** "연산을 통해 도출된 극한값 또는 함수"

Data vs. Parameter

```
# A standard data tensor
x_data = torch.tensor([[1., 2.], [3., 4.]])

# A parameter tensor (we NEED gradients)
w = torch.tensor([[1.0], [2.0]], requires_grad=True)

print(f"Data tensor requires_grad: {x_data.requires_grad}")
print(f"Parameter tensor requires_grad: {w.requires_grad}")
```

Output:

```
Data tensor requires_grad: False
Parameter tensor requires_grad: True
```

IT SENDS A MESSAGE TO THE AUTOGRAD ENGINE:

*"This is a parameter. From now on, track *every single operation* that happens to it."*

AUTOGRAD : LIVE RECORDING OPERATION

Computational Graph

BUILDING THE GRAPH

Goal: Compute $z = x \cdot y$, where $y = a + b$

```
# Three parameter tensors
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
x = torch.tensor(4.0, requires_grad=True)
```

Tensor a=2.0

Tensor b=3.0

Tensor x=4.0

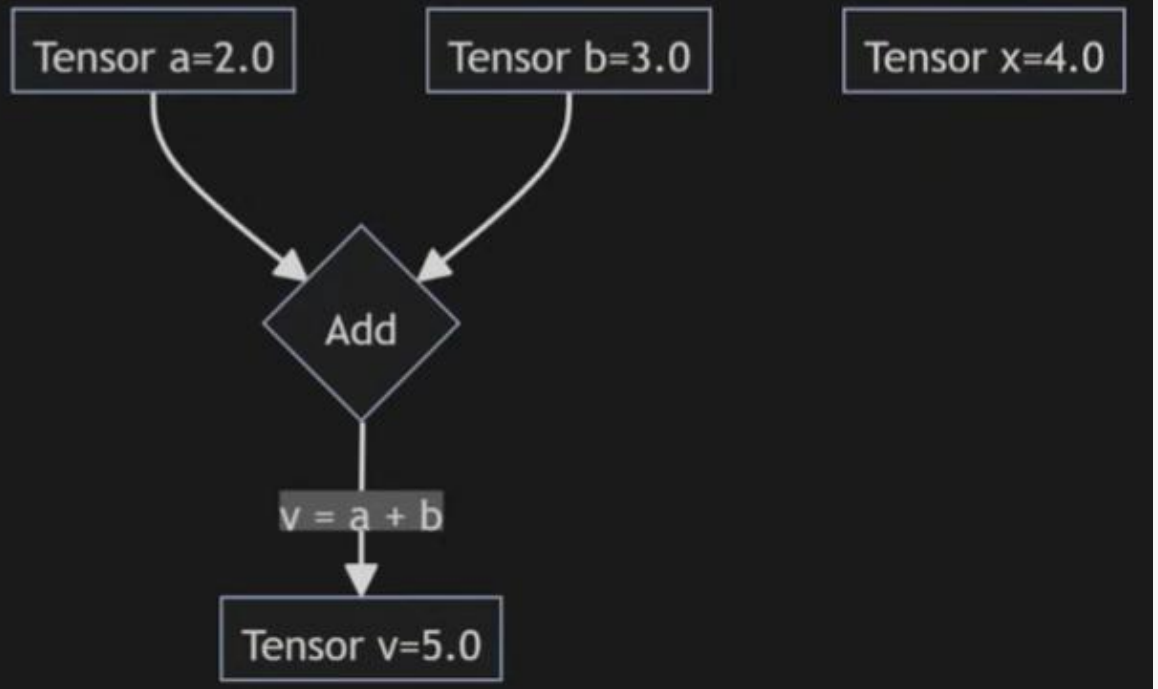
Forward

BUILDING THE GRAPH

Goal: Compute $z = x \cdot y$, where $y = a + b$

```
# Three parameter tensors
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
x = torch.tensor(4.0, requires_grad=True)

y = a + b # First operation
```



Forward

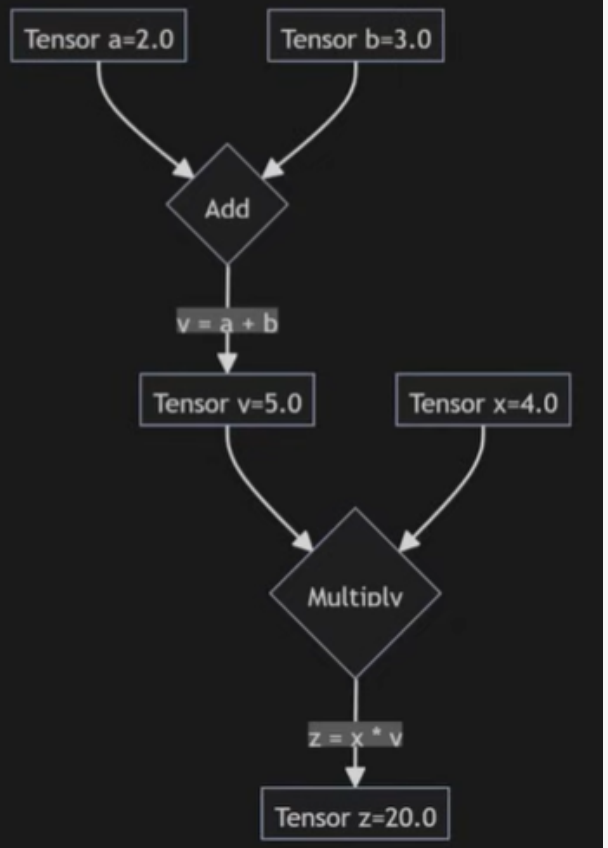
BUILDING THE GRAPH

Goal: Compute $z = x \cdot y$, where $y = a + b$

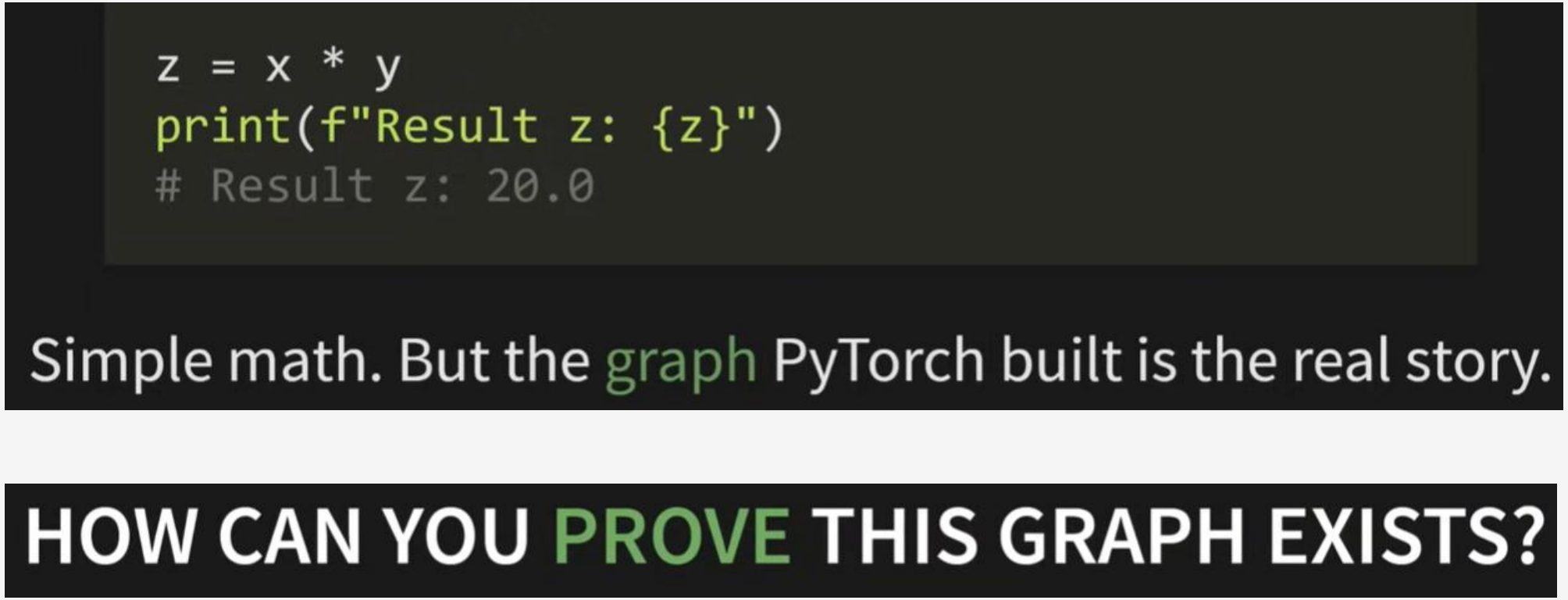
```
# Three parameter tensors
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
x = torch.tensor(4.0, requires_grad=True)

y = a + b # First operation

z = x * y # Second operation
```



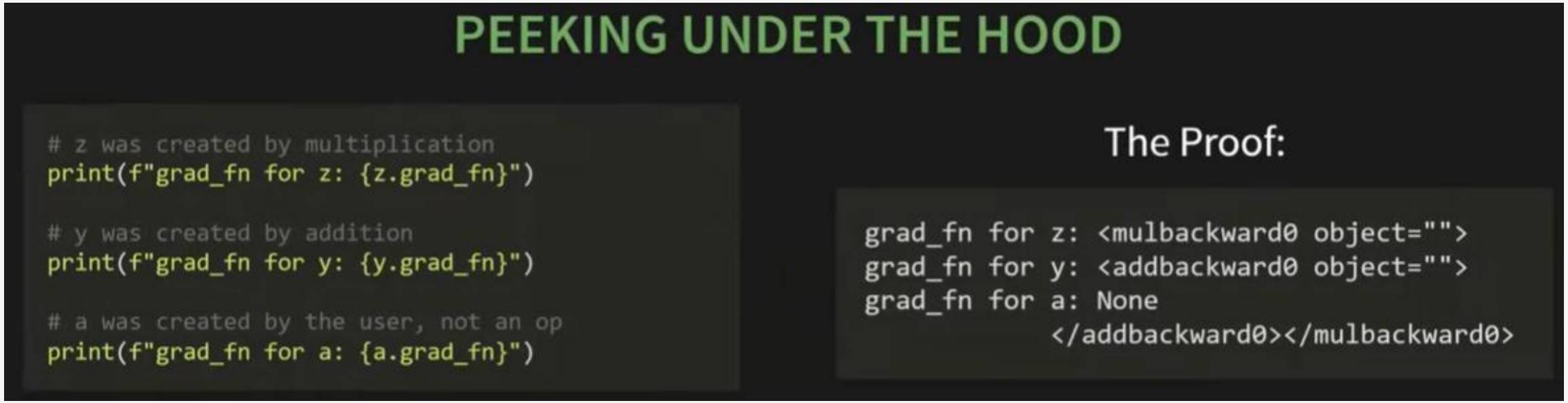
Forward


z = x * y
print(f"Result z: {z}")
# Result z: 20.0
Simple math. But the graph PyTorch built is the real story.
HOW CAN YOU PROVE THIS GRAPH EXISTS?

Backward


EVERY TENSOR CREATED BY AN OPERATION HAS A SPECIAL ATTRIBUTE:
.grad_fn

Backward

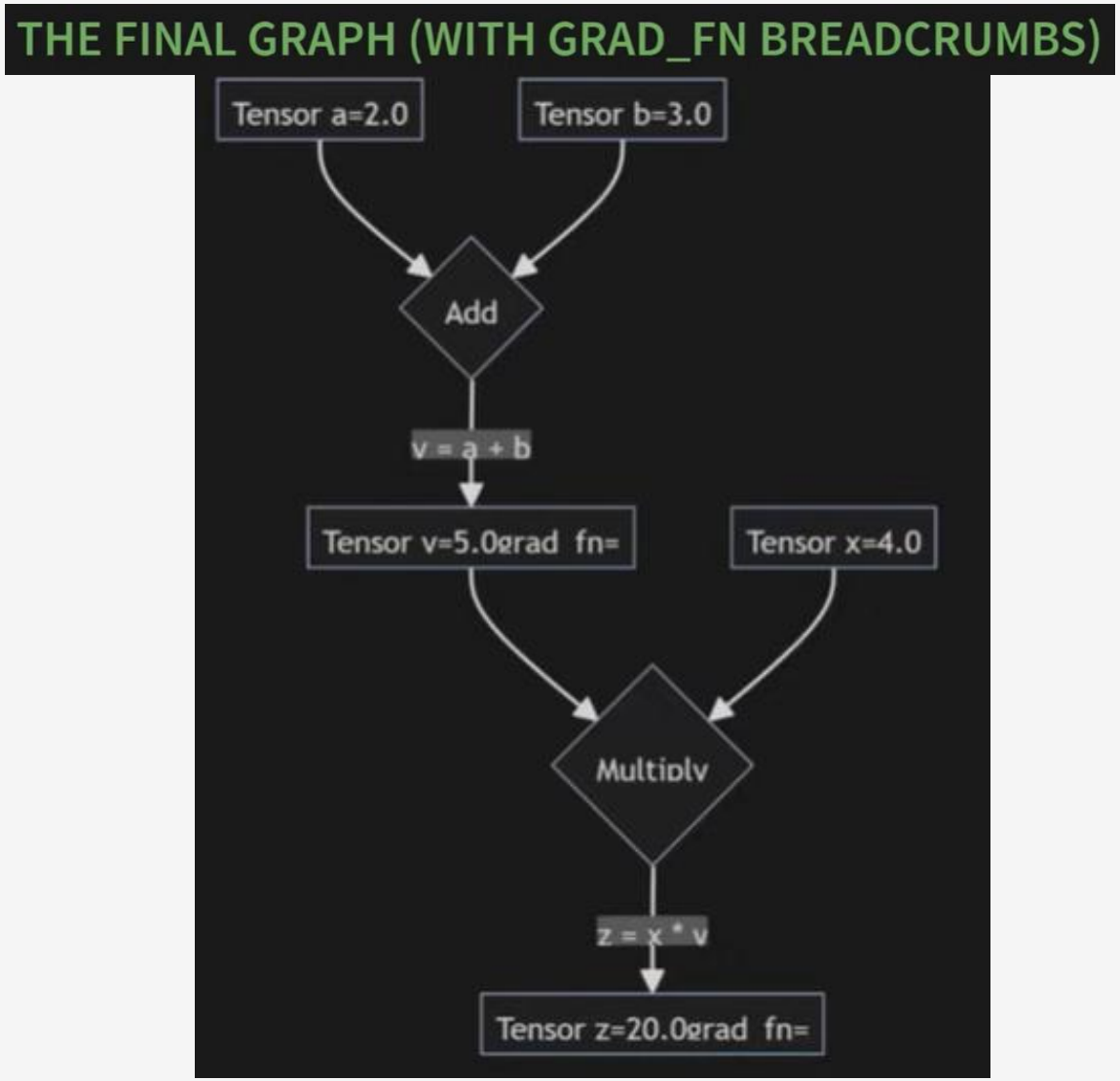

PEEKING UNDER THE HOOD
# z was created by multiplication
print(f"grad_fn for z: {z.grad_fn}")
# y was created by addition
print(f"grad_fn for y: {y.grad_fn}")
# a was created by the user, not an op
print(f"grad_fn for a: {a.grad_fn}")
The Proof:
grad_fn for z: <mulbackward0 object="">
grad_fn for y: <addbackward0 object="">
grad_fn for a: None
</addbackward0></mulbackward0>

`backward()`

THIS IS THE TANGIBLE EVIDENCE OF THE COMPUTATION GRAPH.

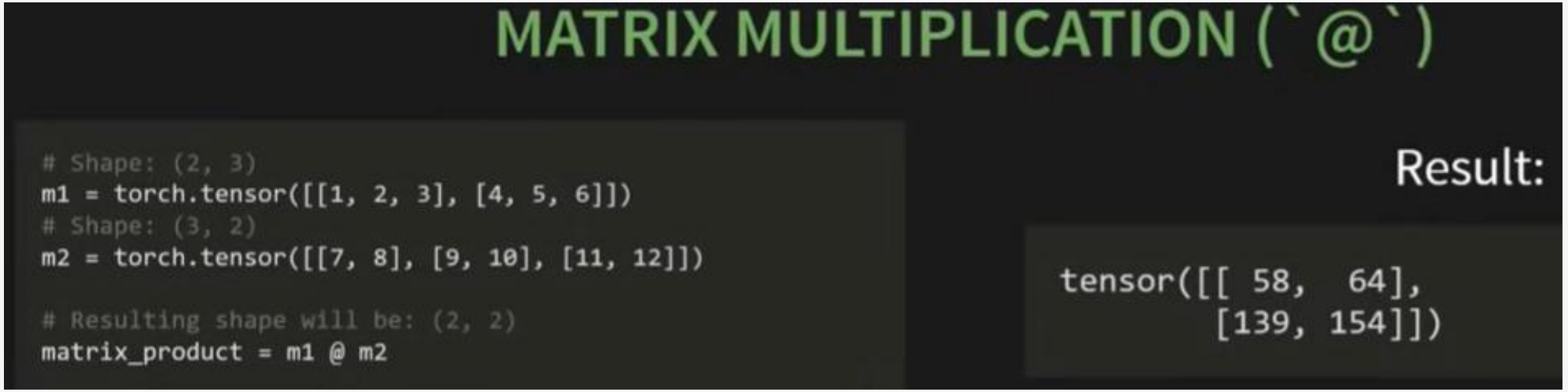
When we call `loss.backward()`, PyTorch will walk this exact trail of breadcrumbs to calculate the gradients.

backward()


THE FINAL GRAPH (WITH GRAD_FN BREADCRUMBS)
Tensor a=2.0
Tensor b=3.0
Add
v = a + b
Tensor v=5.0grad_fn=
Tensor x=4.0
Multiply
z = x * v
Tensor z=20.0grad_fn=

Operations : Inner Product

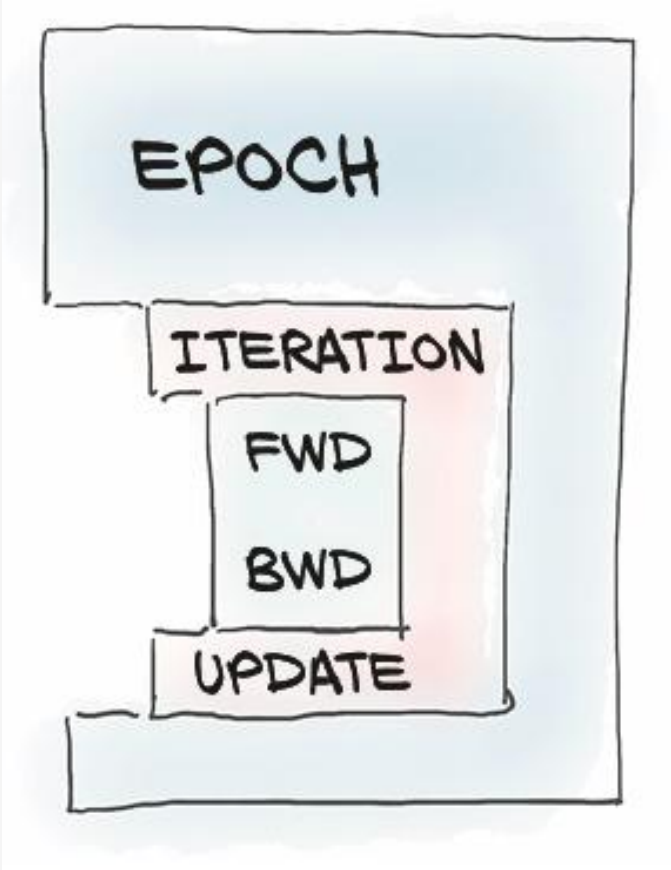
Operations : Inner Product




CHAPTER 5: FROM SCRATCH PART 1
THE FORWARD PASS

5 STEP RECIPE From Scratch

```
FOR N EPOCHS:
  WITH EVERY SAMPLE IN DATASET:
    EVALUATE MODEL (FORWARD)
    COMPUTE LOSS
    COMPUTE GRADIENT OF LOSS
                        (BACKWARD)
    UPDATE MODEL WITH GRADIENT
```



The Forward Pass

1. Prediction

THE 5-STEP DEEP LEARNING RECIPE

Step	Concept (Math)	From Scratch (Raw Tensors)	Professional (`torch.nn`)
1. Prediction	Make a prediction: $\hat{y} = f(X, \theta)$	`y_hat = X @ W + b`	`y_hat = model(X)`
2. Loss Calc	Quantify the error: $L = \text{Loss}(\hat{y}, y)$	`loss = torch.mean((y_hat - y)**2)`	`loss = criterion(y_hat, y)`
3. Gradient Calc	Find the slope of the loss: $\nabla_\theta L$	`loss.backward()`	`loss.backward()`
4. Param Update	Step down the slope: $\theta_{t+1} = \theta_t - \eta \nabla_\theta L$	`W -= lr * W.grad`	`optimizer.step()`
5. Gradient Reset	Reset for the next loop.	`W.grad.zero_()`	`optimizer.zero_grad()`

The Forward Pass

1. Prediction

The Forward Pass

1. Prediction

THE MODEL: SIMPLE LINEAR REGRESSION

$$\hat{y} = XW + b$$

X is our input data, W is the weight, b is the bias.

$\hat{y}$ (y-hat) is our model's prediction, or guess.

The Forward Pass

1. Prediction

W AND b ARE THE TWO "KNOBS" OUR MODEL CAN TURN.

OUR ENTIRE GOAL IS TO FIND THE PERFECT VALUES FOR THEM, SO OUR PREDICTION ŷ GETS AS CLOSE TO THE REAL y AS POSSIBLE.

Generate Data

THE SETUP: CREATING OUR DATA

We'll create fake data that follows the line `y = 2x + 1` with some noise.

```
# Our batch of data will have 10 data points
N = 10
# Each data point has 1 input feature and 1 output value
D_in = 1
D_out = 1

# Create our input data X
X = torch.randn(N, D_in)

# Create our true target labels y by using the "true" W and b
# The "true" W is 2.0, the "true" b is 1.0
true_W = torch.tensor([[2.0]])
true_b = torch.tensor(1.0)
y_true = X @ true_W + true_b + torch.randn(N, D_out) * 0.1 # Add a little noise
```

The Forward Pass

1. Prediction

THE MODEL WILL NEVER SEE `true_W` OR `true_b`.

ITS JOB IS TO DISCOVER THEM JUST BY LOOKING AT X AND `y_true`.

THE PARAMETERS: THE MODEL'S "BRAIN"

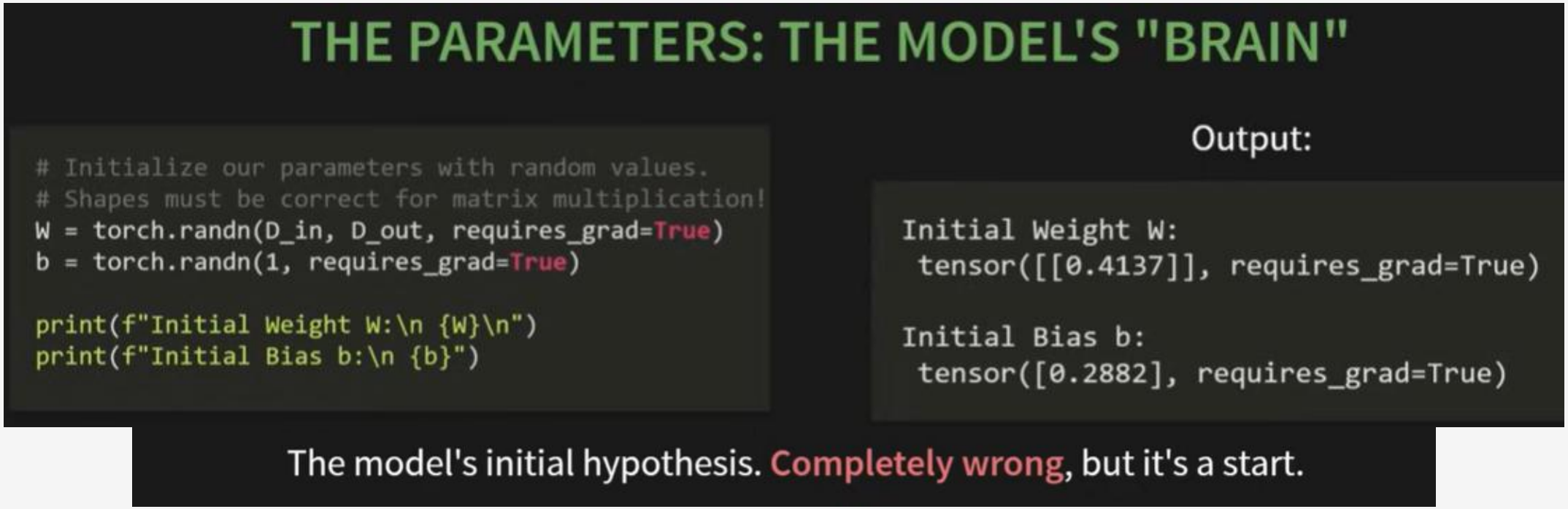
We initialize the model's parameters with random values and turn on the magic switch.

```
# Initialize our parameters with random values.
# Shapes must be correct for matrix multiplication!
W = torch.randn(D_in, D_out, requires_grad=True)
b = torch.randn(1, requires_grad=True)

print(f"Initial Weight W:\n {W}\n")
print(f"Initial Bias b:\n {b}")
```

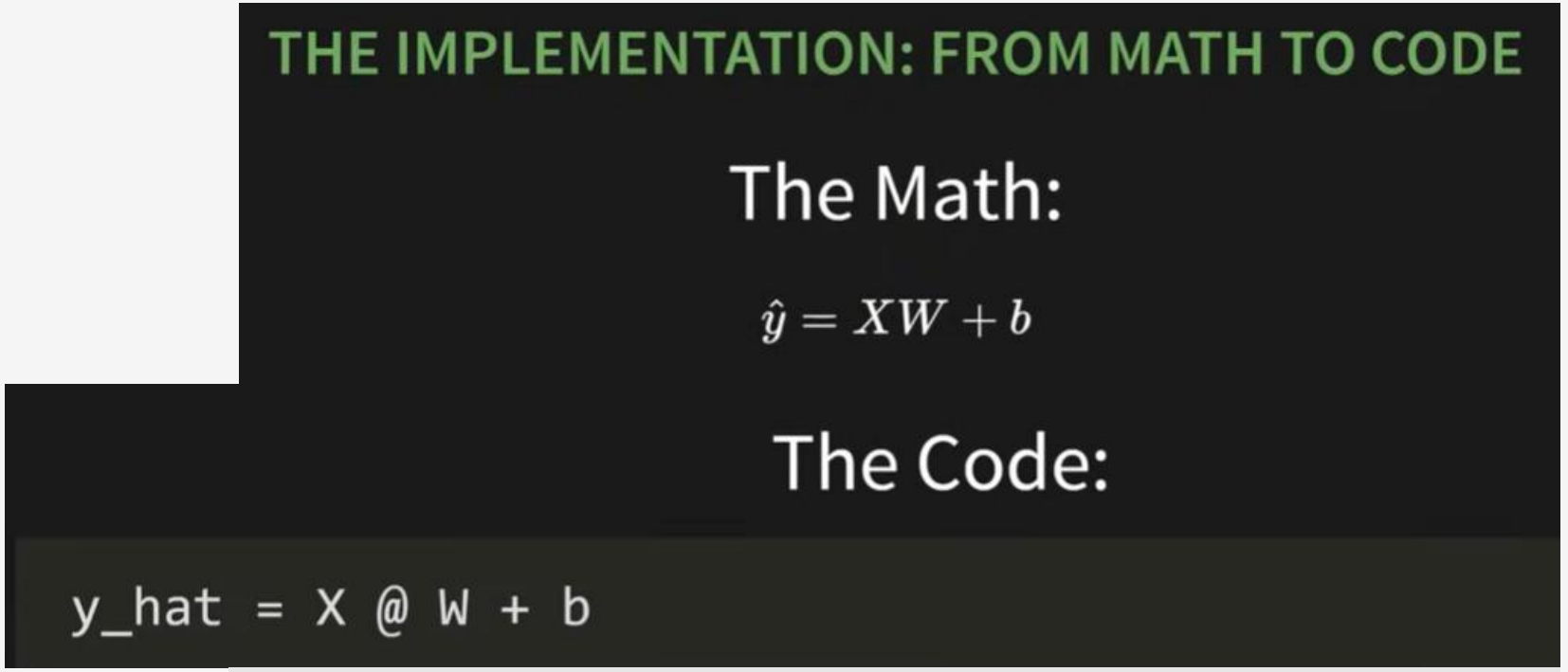
The Forward Pass

1. Prediction



The Forward Pass

1. Prediction



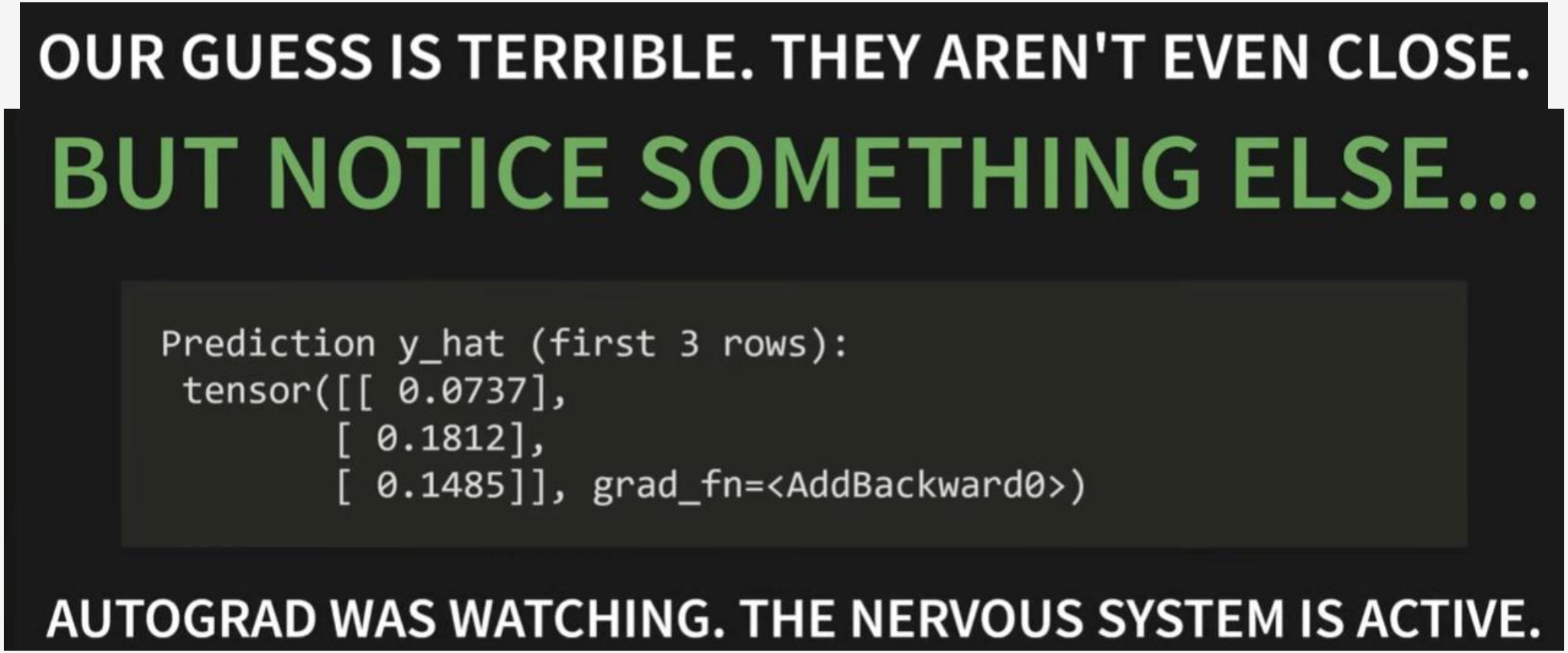
	A	B	C	D	E	F	G	H
1	learning rate		Data		model = y = a+bx			y_hat - y
2	0.01	Iterations	x	y	절편 (α)	기울기 (β)	y_hat	= error
3	1 epoch	0	1	1	0.0000	0.0000	0.0000	-1.0000
4		1	2	3	0.0100	0.0100	0.0300	-2.9700
5		2	4	3	0.0397	0.0694	0.3173	-2.6827
6		3	3	2	0.0665	0.1767	0.5967	-1.4033
7		4	5	5	0.0806	0.2188	1.1746	-3.8254

The Forward Pass

1. Prediction

The Forward Pass

1. Prediction



The Backward Pass

2,3. Gradient Calculation

THIS IS THE BACKWARD PASS. STEP 2 OF OUR LOOP.

IF THE FORWARD PASS WAS THE "GUESS," THE BACKWARD PASS IS THE "POST-MORTEM ANALYSIS."

WE COMPARE OUR GUESS TO THE TRUTH...

...AND THEN WE FIGURE OUT EXACTLY WHO TO BLAME.

THINK OF IT LIKE TUNING A RADIO.

Knob 1: w

Static: $\hat{y}$

Clear Signal: y

Knob 2: b

The backward pass is figuring out **which direction to turn each knob** to make the static go away.

WE NEED A SINGLE NUMBER.

A SCORECARD THAT TELLS US, ON THE WHOLE, HOW BADLY OUR MODEL IS DOING.

WE NEED A SINGLE NUMBER.

A SCORECARD THAT TELLS US, ON THE WHOLE, HOW BADLY OUR MODEL IS DOING.

FOR WHAT WE'RE DOING—REGRESSION—THE MOST COMMON LOSS FUNCTION IS THE...

MEAN SQUARED ERROR (MSE)

THE NAME SOUNDS COMPLICATED, BUT THE IDEA IS SIMPLE.

The Backward Pass

2,3. Gradient Calculation

MEAN SQUARED ERROR (MSE)

$$L = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

Data		model = y = a+bx			y_hat - y	
x	y	절편 (α)	기울기 (β)	y_hat	= error	square(error)
1	1	0.0000	0.0000	0.0000	-1.0000	1.0000
2	3	0.0100	0.0100	0.0300	-2.9700	8.8209
4	3	0.0397	0.0694	0.3173	-2.6827	7.1969
3	2	0.0665	0.1767	0.5967	-1.4033	1.9694
5	5	0.0806	0.2188	1.1746	-3.8254	14.6337

In plain English:

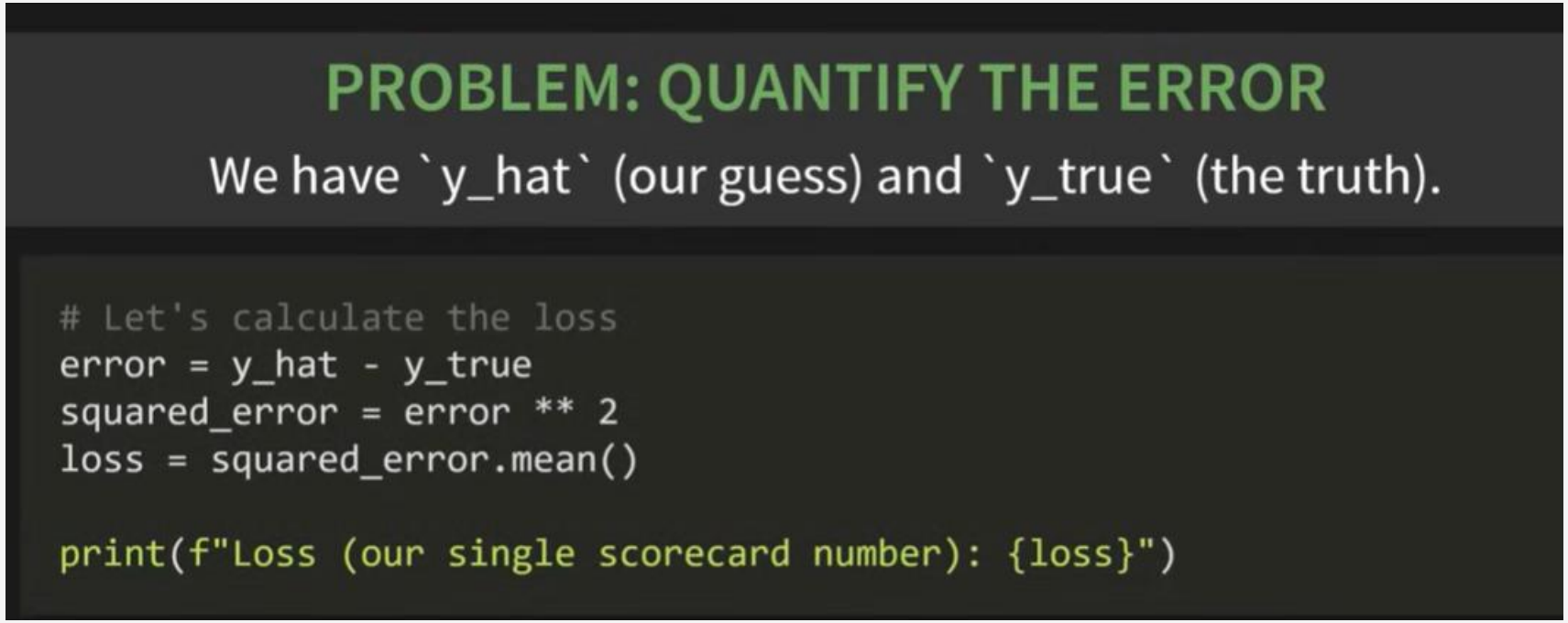
For every prediction, find the difference: $(\hat{y} - y)$

Square that difference to make it positive: $(\hat{y} - y)^2$

Take the average of all those squared differences.

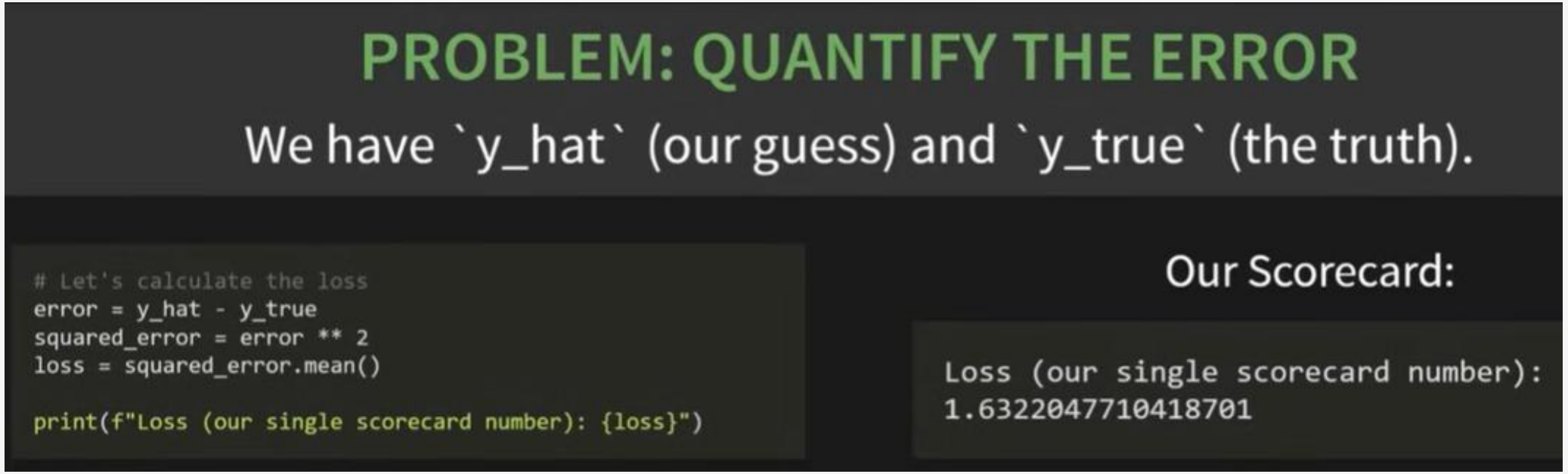
The Backward Pass

2,3. Gradient Calculation



The Backward Pass

2,3. Gradient Calculation



NOTICE THAT THIS `LOSS` TENSOR ALSO HAS A `GRAD_FN`.

It knows it's the result of all the previous calculations. It sits at the very end of our computation graph.

NOW FOR THE MAGIC.

The Backward Pass

2,3. Gradient Calculation

HOW DO WE KNOW WHICH WAY TO TURN THE `W` AND `B` KNOBS TO LOWER THAT SCORE?
AUTOGRAD DOES ALL THE HEAVY LIFTING FOR US. WITH ONE SINGLE COMMAND.

THIS MIGHT BE THE SINGLE MOST POWERFUL LINE OF CODE IN ALL OF PYTORCH.

WE ARE TELLING PYTORCH:

"Travel backward from `loss` and calculate gradients for all parameters with `requires_grad=True`."

The Backward Pass

2,3. Gradient Calculation

IN OUR CASE, IT WILL COMPUTE THE TWO MOST IMPORTANT VALUES WE NEED:

$\frac{\partial L}{\partial W}$: The gradient of the Loss w.r.t. our Weight `W`.

$\frac{\partial L}{\partial b}$: The gradient of the Loss w.r.t. our Bias `b`.

READY? HERE IS THE COMMAND.

```
loss.backward()
```


PYTORCH JUST PERFORMED THE MOST CRITICAL STEP OF THE LEARNING PROCESS.
IT POPULATED A HIDDEN ATTRIBUTE FOR OUR `W` AND `B` TENSORS.
THE .grad ATTRIBUTE.

The Backward Pass

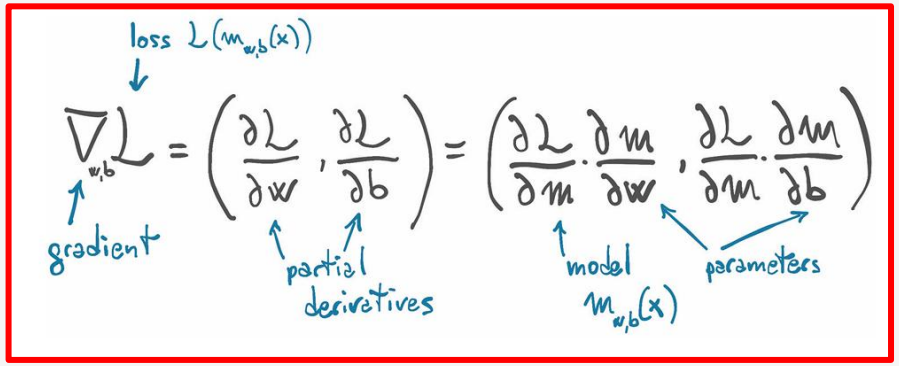
2,3. Gradient Calculation

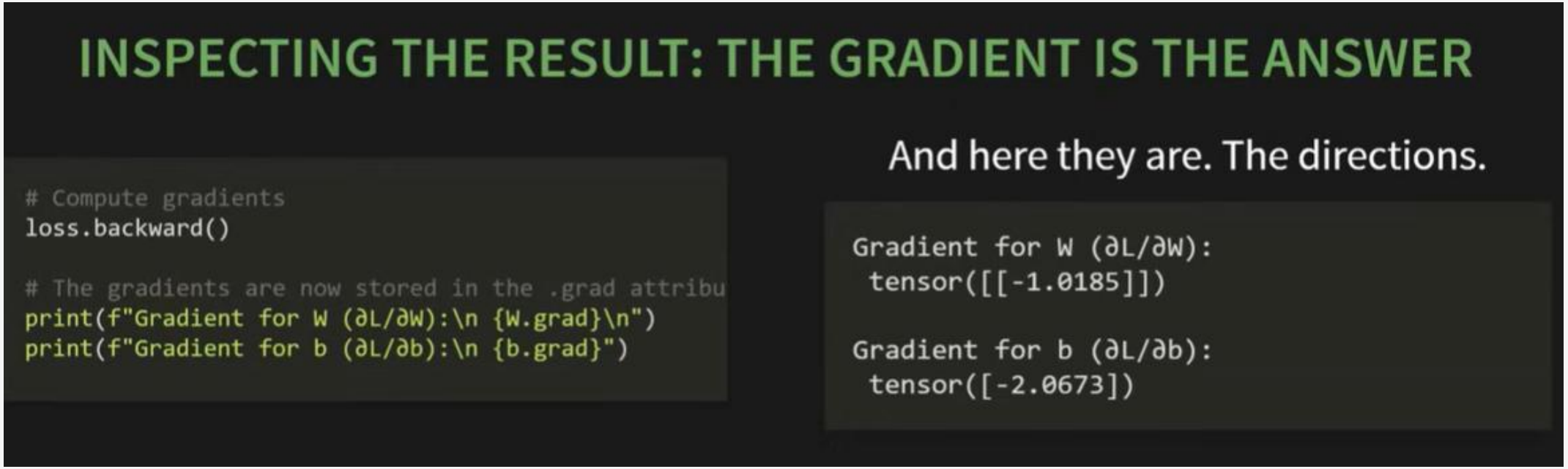
INSPECTING THE RESULT: THE GRADIENT IS THE ANSWER

The .grad attribute now holds the "signal" that tells us exactly how to adjust our knobs.

```
# Compute gradients
loss.backward()

# The gradients are now stored in the .grad attribute
print(f"Gradient for W (∂L/∂W):\n {W.grad}\n")
print(f"Gradient for b (∂L/∂b):\n {b.grad}")
```




INSPECTING THE RESULT: THE GRADIENT IS THE ANSWER
# Compute gradients
loss.backward()
# The gradients are now stored in the .grad attribu
print(f"Gradient for W (∂L/∂W):\n {W.grad}\n")
print(f"Gradient for b (∂L/∂b):\n {b.grad}")
And here they are. The directions.
Gradient for W (∂L/∂W):
tensor([[-1.0185]])
Gradient for b (∂L/∂b):
tensor([-2.0673])

The Backward Pass

2,3. Gradient Calculation

HOW DO WE KNOW WHICH WAY TO TURN THE `W` AND `B` KNOBS TO LOWER THAT SCORE?

AUTOGRAD DOES ALL THE HEAVY LIFTING FOR US. WITH ONE SINGLE COMMAND.

`W.grad` is -1.0185: The sign is everything.

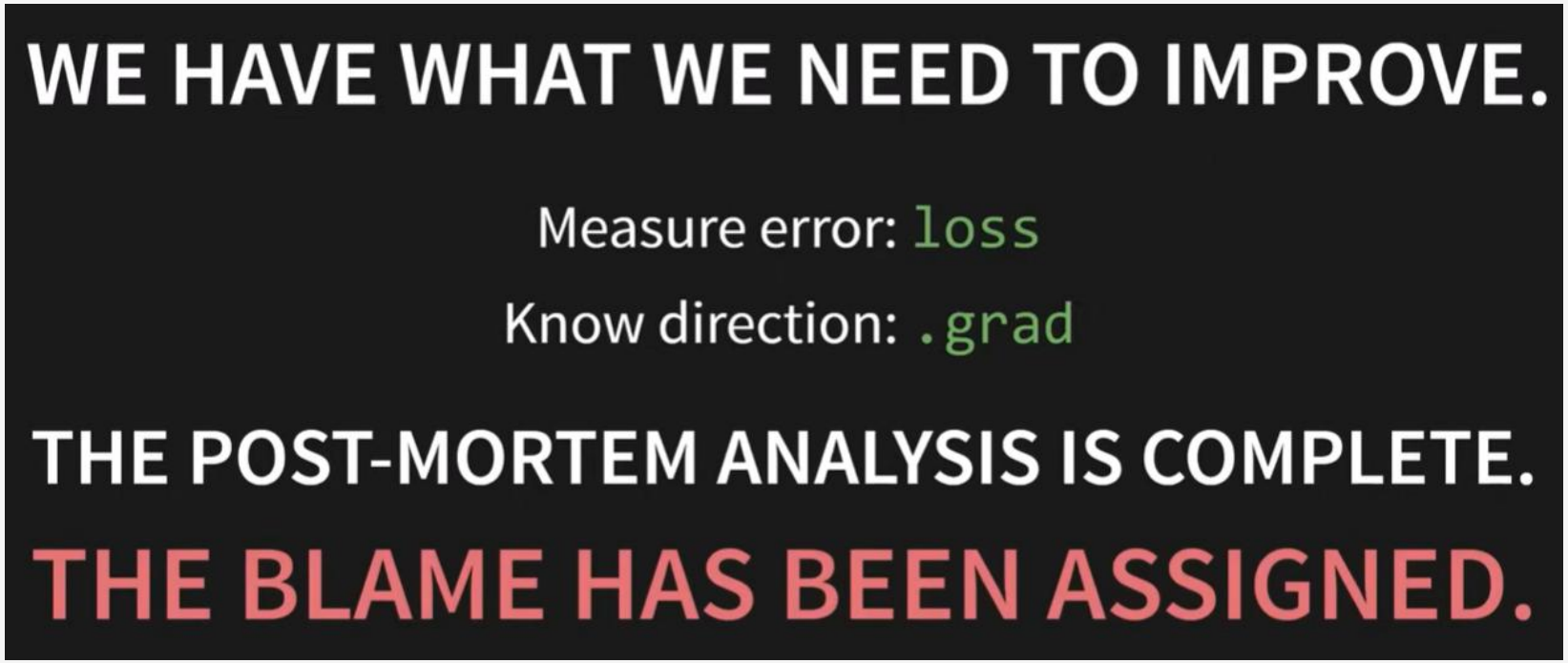
Negative gradient → increasing `W` decreases loss.

Gradient points toward steepest INCREASE.

b.grad IS -2.0673: IT'S THE SAME STORY.

	A	B	C	D	E	F	G	H
1	learning rate		Data		model = y = a+bx			y_hat - y
2	0.01	Iterations	x	y	절편 (α)	기울기 (β)	y_hat	= error
3	1 epoch	0	1	1	0.0000	0.0000	0.0000	1.0000
4		1	2	3	0.0100	0.0100	0.0300	-2.9700
5		2	4	3	0.0397	0.0694	0.3173	-2.6827
6		3	3	2	0.0665	0.1767	0.5967	-1.4033
7		4	5	5	0.0806	0.2188	1.1746	-3.8254

Larger magnitude = steeper slope.


WE HAVE WHAT WE NEED TO IMPROVE.
Measure error: loss
Know direction: .grad
THE POST-MORTEM ANALYSIS IS COMPLETE.
THE BLAME HAS BEEN ASSIGNED.

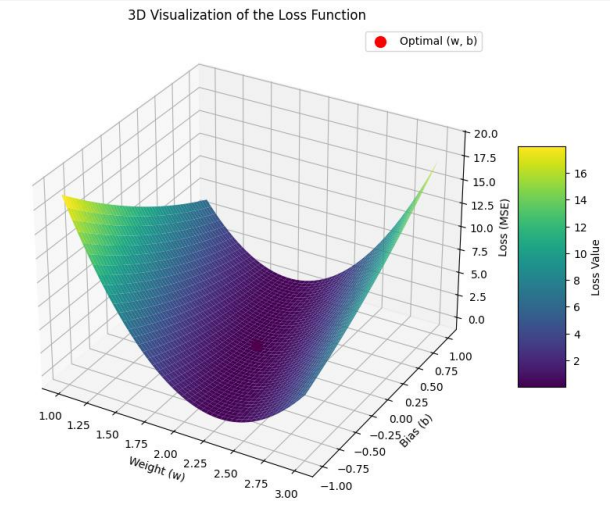
The Backward Pass

2,3. Gradient Calculation

WE ARE STANDING ON A FOGGY MOUNTAIN.

THE loss TELLS US OUR CURRENT ALTITUDE.

THE gradients TELL US THE DIRECTION OF THE STEEPEST UPHILL SLOPE.



SO WHAT DO WE DO?

WE TAKE A SMALL STEP DOWNHILL.

	A	B	C	D	E	F	G	H
1	learning rate		Data		model = y = a+bx			y_hat - y
2	0.01	Iterations	x	y	절편 (α)	기울기 (β)	y_hat	= error
3	1 epoch	0	1	1	0.0000	0.0000	0.0000	-1.0000
4		1	2	3	0.0100	0.0100	0.0300	-2.9700
5		2	4	3	0.0397	0.0694	0.3173	-2.6827
6		3	3	2	0.0665	0.1767	0.5967	-1.4033
7		4	5	5	0.0806	0.2188	1.1746	-3.8254

THIS IS WHERE LEARNING ACTUALLY HAPPENS, THROUGH A SIMPLE AND BEAUTIFUL ALGORITHM CALLED...

The Backward Pass

2,3. Gradient Calculation

THE SINGLE MOST IMPORTANT FORMULA

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta L$$

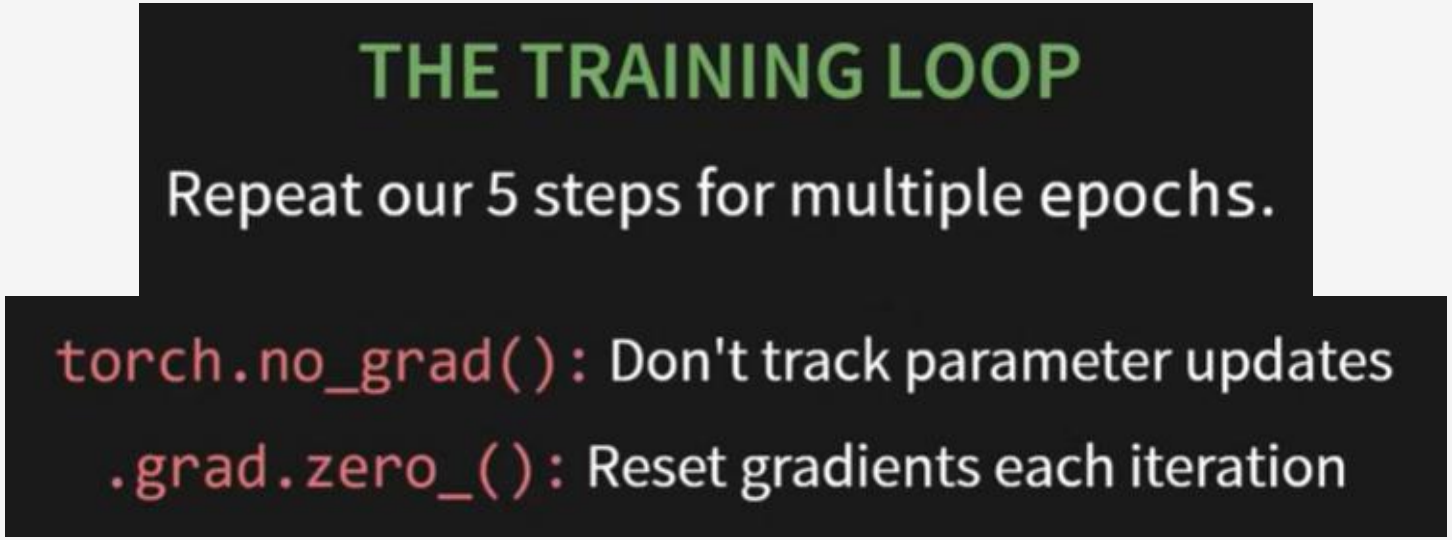
- θ (theta) represents all our parameters. For us, that's `W` and `b`.
- η (eta) is the learning rate. A small number (e.g., 0.01) controlling our step size.
- $\nabla_\theta L$ is the gradient of the loss. We just calculated this! It's in `W.grad` and `b.grad`.

THE UPDATE RULES IN CODE

```
W_new = W_old - learning_rate * W.grad
b_new = b_old - learning_rate * b.grad
```

We are literally just nudging our parameters in the opposite direction of the gradient.

4,5. Parameter Update & reset



Gradient Accumulation : PyTorch의 디자인 철학은 loss.backward()를 호출할 때마다 기울기를 덮어쓰는 것이 아니라 기존 값에 더하도록 되어 있기 때문에 만약 리셋을 안 하면 다음과 같이 작동하여 학습(훈련)이 안됨

- **1회차:** $grad = g_1$
- **2회차:** $grad = g_1 + g_2$
- **3회차:** $grad = g_1 + g_2 + g_3$

The Training Loop

4,5. Parameter Update & reset

FOR N EPOCHS:

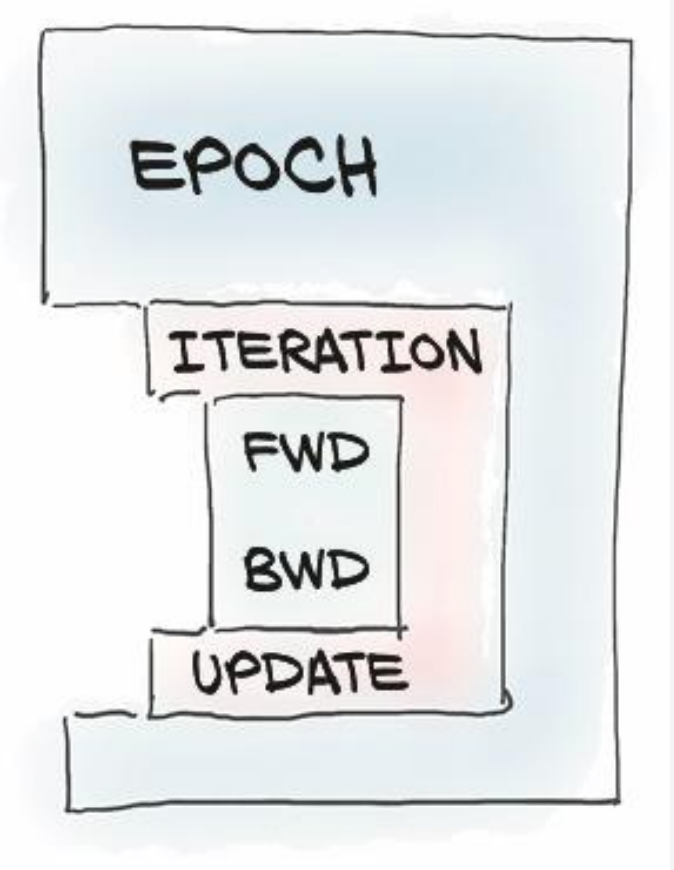
  WITH EVERY SAMPLE IN DATASET:

    EVALUATE MODEL (FORWARD)

    COMPUTE LOSS

    COMPUTE GRADIENT OF LOSS (BACKWARD)

    UPDATE MODEL WITH GRADIENT



The Entire Training Process — 4,5. Parameter Update & reset

```
# Hyperparameters
learning_rate, epochs = 0.01, 100
# Re-initialize parameters
W, b = torch.randn(1, 1, requires_grad=True), torch.randn(1, requires_grad=True)

# Training Loop
for epoch in range(epochs):
    # Forward pass and loss
    y_hat = X @ W + b
    loss = torch.mean((y_hat - y_true)**2)

    # Backward pass
    loss.backward()

    # Update parameters
    with torch.no_grad():
        W -= learning_rate * W.grad; b -= learning_rate * b.grad

    # Zero gradients
    W.grad.zero_(); b.grad.zero_()
```

Steps 1 & 2: Make a guess and calculate the error.

Step 3: Calculate the "blame" for the error.

Step 4: Nudge the parameters in the correct direction.

Step 5: Reset for the next round of learning.

4,5. Parameter Update & reset

WATCHING THE MODEL LEARN

Let's add a print statement inside that loop and watch what happens.

```
# ... (inside the loop)
    if epoch % 10 == 0:
        print(f"Epoch {epoch:02d}: Loss={loss.item():.4f}, W={W.item():.3f}, b={b.item():.3f}")

print(f"\nFinal Parameters: W={W.item():.3f}, b={b.item():.3f}")
print(f"True Parameters:  W=2.000, b=1.000")
```

The Training Loop

4,5. Parameter Update & reset

LET'S RUN IT.

Epoch	Loss	W	b
00	4.1451	-0.347	0.505
10	1.0454	0.485	0.887
20	0.2917	0.970	1.077
30	0.1068	1.251	1.155
40	0.0592	1.422	1.178
50	0.0441	1.528	1.178
60	0.0381	1.600	1.168
70	0.0354	1.650	1.154
80	0.0339	1.685	1.140
90	0.0329	1.711	1.127

Optimizer

```
import torch.optim as optim

# 1. 설정 및 데이터 준비
learning_rate, epochs = 0.01, 100
W, b = torch.randn(1, 1, requires_grad=True), torch.randn(1, requires_grad=True)

# Optimizer 정의 (여기서는 확률적 경사하강법 SGD 사용)
optimizer = torch.optim.SGD([W, b], lr=learning_rate)

# Training Loop
for epoch in range(epochs):
    # Forward pass
    y_hat = X @ W + b
    loss = torch.mean((y_hat - y_true)**2)

    # --- 핵심 세 줄 (The Three Lines) ---
    optimizer.zero_grad()  # 1. 기울기 초기화
    loss.backward()        # 2. 기울기 계산 (Backpropagation)
    optimizer.step()       # 3. 파라미터 업데이트
```

```
# 현재 학습된 가중치(W)와 편향(b)
print(f"Weights (W): {W.data}“)
print(f"Bias (b): {b.data}")
# 미분 가능한지 정보
print(W, b)
```

model (like pro)

```
import torch
import torch.nn as nn
import torch.optim as optim

# 모델 클래스 정의 (nn.Module 상속)
class LinearRegressionModel(nn.Module):
    def __init__(self):
        super(LinearRegressionModel, self).__init__()
        # W, b를 자동 생성 Linear 레이어(입력1, 출력1)
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        # 1단계: 예측 (y_hat = X @ W + b)
        return self.linear(x)

learning_rate = 0.01; epochs = 100

model = LinearRegressionModel()

# 손실 함수 정의 (Mean Squared Error)
criterion = nn.MSELoss()
```

```
# 옵티마이저 정의 (SGD)
optimizer = optim.SGD(model.parameters(),
lr=learning_rate)

# 학습 루프 (Training Loop)
for epoch in range(epochs):
    optimizer.zero_grad()      # 기울기 초기화
    y_hat = model(X)           # 모델을 통한 예측
    loss = criterion(y_hat, y_true)      # 손실 계산
    loss.backward()            # 역전파(기울기 계산)
    optimizer.step()           # 파라미터 업데이트

    if (epoch + 1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss:
{loss.item():.4f}')

# 모델 저장 및 확인
print("\n[학습된 파라미터 확인]")
print(model.state_dict())

torch.save(model.state_dict(), 'model.pth')
```

pytorch tutorial

1. Installation
2. Tensor Basics
3. Autograd
4. Backpropagation
5. Gradient Descent With Autograd and Backpropagation
6. Training Pipeline: Model, Loss, and Optimizer
7. Linear Regression
8. Logistic Regression
9. Dataset and DataLoader
10. Dataset Transforms
11. Softmax And Cross Entropy
12. Activation Functions
13. Feed-Forward Neural Net
14. Convolutional Neural Net (CNN)
15. Transfer Learning
16. Tensorboard
17. Save and Load Models

pytorch